

The System Design Handbook

— Midhun Soma

A production-grade reference for engineers, from fundamentals to FAANG-level interviews

Note: Every chapter follows the same skeleton — Definition → Why it exists → Problem it solves → Internal working → Architecture/diagram → Example → Advantages/Disadvantages → Real production examples → Interview questions → Common mistakes → Best practices → Summary. Where a chapter's content already covers an item naturally, it isn't repeated as a separate bullet — depth over checkbox-filling. Chapter 19's system walkthroughs go to full depth for four flagship systems (Chat, Video Streaming, Notifications, plus a condensed URL Shortener) and give a repeatable framework plus a focused paragraph for the rest, rather than shallow coverage of all twelve spread thin.

Table of Contents

Part I — Foundations

1. Introduction to System Design
2. Requirements Gathering
3. Scaling
4. Load Balancing
5. API Gateway
6. Rate Limiting
7. Middleware

Part II — Distributed Data

8. CAP Theorem
9. Caching
10. Database Scaling
11. SQL vs NoSQL
12. Message Queues
13. Monolith vs Microservices

Part III — Scale, Resilience & Applied Design

14. Consistent Hashing
15. CDN Internals
16. Async Programming
17. Observability
18. System Design Patterns
19. Design a Large-Scale System

Chapter 1: Introduction to System Design

1.1 Definition

System design is the discipline of defining the architecture, components, modules, interfaces, and data flow of a software system to satisfy specified functional and non-functional requirements — at a given scale, cost, and reliability target.

Two distinct sub-disciplines fall under this umbrella:

- **High-Level Design (HLD)** — the macro architecture: which services exist, how they communicate, where data lives, how traffic flows. This is what “system design interviews” actually test.
- **Low-Level Design (LLD)** — the micro architecture: class diagrams, design patterns, function signatures, database schemas at the table/field level. This is closer to traditional software engineering.

1.2 Why it exists / problem it solves

Writing correct code answers “does this function do what it’s supposed to?” It does not answer:

- What happens when 50,000 people hit this endpoint in the same second?
- What happens when the database server dies mid-write?
- What happens when a network partition splits your servers into two groups that can’t talk to each other?
- How do you deploy a new version without downtime?
- How do you know something is wrong *before* a user tells you?

Before system design existed as an explicit discipline, engineers built systems that worked in the demo and collapsed in production. System design is the accumulated set of patterns the industry converged on after two decades of exactly that failure mode repeating at scale.

1.3 Client-Server Architecture

Nearly every system you’ll design is a variation on client-server: a client (browser, mobile app, another service) sends a request over a network to a server, which processes it and returns a response.

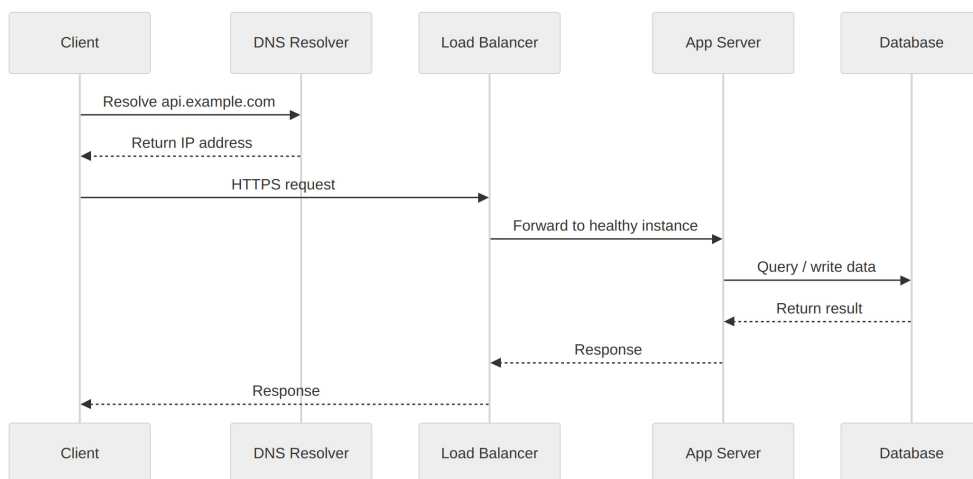


Diagram 1

This single diagram is the skeleton every later chapter attaches to. DNS resolution happens once (then gets cached by the OS/browser). Every subsequent chapter — load balancing, gateways, caching, databases — is really just “what happens between steps 3 and 6, in more detail.”

1.4 Stateless vs Stateful services

This distinction determines almost every scaling decision you’ll make later, so it deserves its own emphasis here.

- **Stateless service:** doesn't retain any client-specific data between requests. Every request carries everything the server needs (typically via a token). Any instance can handle any request. This is what makes horizontal scaling trivial — you can add or remove servers freely because none of them are “special.”
- **Stateful service:** retains data specific to a session or client between requests (e.g. a game server holding live match state in memory, a WebSocket connection). Scaling these requires sticky sessions or externalizing the state (e.g. to Redis) — much harder.

Best practice: Design application servers to be stateless wherever possible. Push state into the database or a shared cache (Redis). This one decision is what makes horizontal scaling, load balancing, and zero-downtime deploys straightforward instead of painful.

1.5 Why companies ask this in interviews

A DSA interview tests whether you can solve a well-defined problem correctly and efficiently. A system design interview tests something DSA cannot: **whether you can operate under ambiguity, make and defend trade-offs, and reason about a system that doesn't have one correct answer.** At senior levels, this is a much stronger predictor of on-the-job performance than algorithmic puzzle-solving — which is exactly why it dominates loop interviews from mid-level upward.

Chapter 1 Summary

- System design = HLD (architecture) + LLD (implementation-level design)
- Client-server + DNS + load balancer + app server + database is the skeleton of nearly every system
- Stateless design is the single biggest enabler of easy horizontal scaling
- Interviews test judgment and trade-off reasoning, not memorized “correct” architectures

Interview cheat sheet

- Always clarify HLD vs LLD scope before diving in — interviewers often mean HLD only
- Say “stateless” explicitly when describing your app tier — it signals you understand scaling
- Never present an architecture as “the” answer — frame it as “given these constraints, here's what I'd choose, and here's the trade-off”

Common mistakes

- Jumping straight to drawing boxes without clarifying requirements first (Chapter 2 exists for a reason)
 - Designing a stateful app tier by default out of habit from smaller projects
-

Chapter 2: Requirements Gathering

2.1 Definition

Before any architecture decision, a system design conversation must establish **what** the system does (functional) and **how well** it needs to do it (non-functional). Skipping this step is the single most common reason system design interviews go badly — candidates design for imagined constraints instead of stated ones.

2.2 Functional Requirements

Concrete, testable capabilities. “A user can upload a video.” “A user can send a message to another user.” These come directly from the product spec and are usually the easy part of the conversation.

2.3 Non-Functional Requirements (NFRs)

This is where real engineering judgment lives. Each NFR pulls your design in a different direction, and they frequently conflict with each other — the entire craft of system design is resolving those conflicts deliberately.

NFR	What it means	Example question it answers
Availability	System responds successfully, even during failures	“What % of requests succeed in a year?”
Latency	Time for a single request to complete	“How fast does the user see a result?”
Consistency	All nodes see the same data at the same time	“If I update my profile, does my friend see it instantly?”
Durability	Data survives after being acknowledged as saved	“If the disk dies, is the write gone?”
Scalability	System handles growth in load/data	“What happens at 100x current traffic?”
Reliability	System performs correctly over time without failure	“Does it work the same way every time?”
Fault Tolerance	System keeps functioning despite component failure	“One server dies — does the user notice?”
Maintainability	Ease of fixing bugs / operating the system	“Can an on-call engineer debug this at 3 AM?”
Extensibility	Ease of adding new features	“Can we add a new payment provider without a rewrite?”
Observability	Ability to understand internal state from outputs	“Can we tell <i>why</i> it’s slow, not just <i>that</i> it’s slow?”
Security	Protection against unauthorized access	“Can an attacker read another user’s data?”
Privacy / Compliance	Legal and regulatory data handling	“Is user data stored in the required region (GDPR)?”

Warning: Availability and Consistency are in direct tension under network partitions — this tension is formalized as the CAP theorem, covered fully in Chapter 8. Every distributed system you design is implicitly choosing a position on this trade-off, whether you state it or not.

2.4 Availability, quantified

Availability is usually expressed in “nines”:

Availability	Downtime per year
99% (two nines)	~3.65 days
99.9% (three nines)	~8.76 hours
99.99% (four nines)	~52.6 minutes
99.999% (five nines)	~5.26 minutes

Each additional nine costs disproportionately more engineering effort — redundancy, automated failover, multi-region deployment. Part of requirements gathering is agreeing *which* nine you actually need, because five-nines infrastructure for an internal admin tool is wasted spend.

2.5 Worked example: URL shortener

A quick illustration of turning a vague prompt into requirements:

Functional: Create a short URL from a long one. Redirect a short URL to the original. Optionally: custom aliases, expiration, analytics.

Non-functional: - High availability (a broken redirect breaks every link already shared) - Low latency on redirect (~sub-100ms — this is the hot path) - Reads vastly outnumber writes (typically 100:1 or more) — this single fact drives the caching strategy in Chapter 9 - Eventual consistency is acceptable for analytics counts; strong consistency is required for the short-code-to-URL mapping itself

Chapter 2 Summary

- Functional requirements = what it does; non-functional = how well
- NFRs conflict with each other by design — the job is choosing which to prioritize, not satisfying all of them equally
- Quantify availability in nines and consistency in “how stale is acceptable” — vague answers (“it should be fast and reliable”) don’t hold up in real design

Interview cheat sheet

- Spend the first 3-5 minutes of any system design interview purely on requirements — resist the urge to draw boxes early
- Always ask about read:write ratio — it’s the single most architecture-determining number in most systems
- State your NFR priorities explicitly (“I’m prioritizing availability over strong consistency here because...”)

Common mistakes

- Treating all NFRs as equally important instead of ranking them for the specific system
- Forgetting to size the system (users, requests/sec, data volume) before proposing architecture

Chapter 3: Scaling

3.1 Definition

Scaling is the process of increasing a system's capacity to handle more load — more users, more requests, more data — without a proportional degradation in performance.

3.2 Vertical Scaling (Scale Up)

Adding more resources (CPU, RAM, disk) to a single machine.

Advantages: simplest possible change (often zero code changes), no distributed-systems complexity, no data consistency issues since there's only one node.

Disadvantages: hard physical ceiling (you can't buy an infinitely large machine), single point of failure, expensive at the high end (large machines cost disproportionately more per unit of capacity), requires downtime to resize in most cloud setups.

3.3 Horizontal Scaling (Scale Out)

Adding more machines and distributing load across them.

Advantages: near-limitless ceiling, no single point of failure (one node dying doesn't take down the system), commodity hardware is cheaper per unit than high-end single machines, enables zero-downtime deploys (roll servers one at a time).

Disadvantages: introduces real distributed-systems complexity — you now need a load balancer, you need to handle data consistency across nodes, you need service discovery, network calls between nodes add latency and failure modes that didn't exist before.

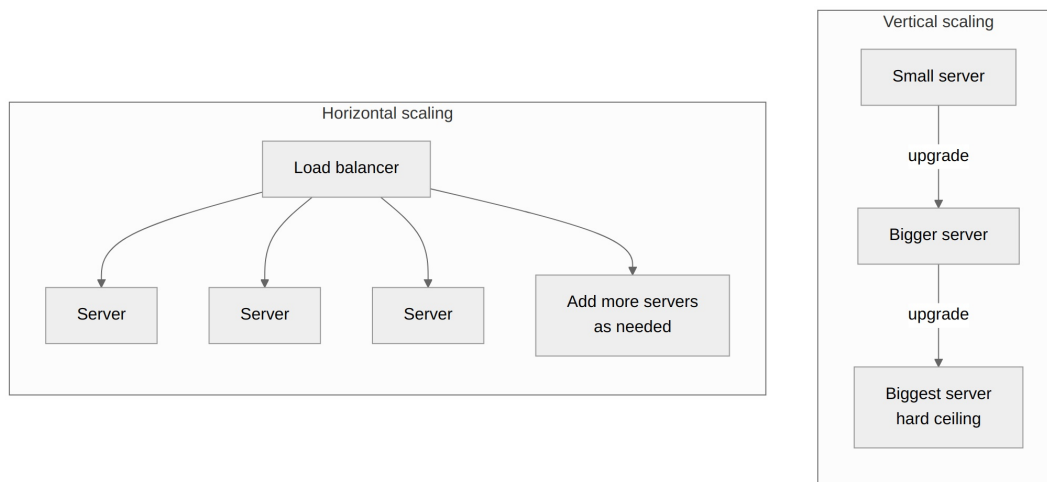


Diagram 2

3.4 Scaling bottlenecks

Every system eventually bottlenecks on one of four resources. Knowing which one you're hitting determines the fix:

Bottleneck	Symptom	Typical fix
CPU	High latency under load, low idle time	Horizontal scale, optimize hot code paths, offload work to async jobs
Memory	OOM kills, swapping, GC pauses	Vertical scale, reduce per-request memory footprint, add caching layer to reduce recomputation
Disk I/O	Slow database writes/reads, high disk queue depth	SSD/NVMe upgrade, read replicas, better indexing, move hot data to cache
Network	Timeouts, high latency between services	Reduce chattiness between services, batch requests, move services closer (same AZ/region), use a CDN for static assets

Tip: Always identify *which* resource is actually the bottleneck before scaling. Vertically scaling a CPU-bound service when the real bottleneck is unindexed database queries just moves the same problem to a more expensive machine.

3.5 Real-world pattern

Most production systems use **both**: individual database nodes are scaled vertically up to a practical ceiling (databases are the hardest layer to horizontally scale — see Chapter 10 on sharding), while the stateless application tier scales horizontally almost without limit, since it costs nothing but infrastructure to add another identical instance behind a load balancer.

Chapter 3 Summary

- Vertical = bigger machine, simple but capped. Horizontal = more machines, unlimited but complex.
- Statelessness (Chapter 1) is the prerequisite that makes horizontal scaling easy
- Always diagnose the actual bottleneck (CPU/memory/disk/network) before choosing a scaling strategy

Interview cheat sheet

- Default answer for app tier: horizontal. Default answer for a single relational database: vertical first, then read replicas, then sharding as a last resort (Chapter 10)
- Mention statelessness explicitly when proposing horizontal scaling — it’s the enabling assumption

Common mistakes

- Assuming horizontal scaling is “always better” — it introduces real complexity that isn’t worth it for low-traffic systems
- Forgetting that databases don’t horizontally scale as easily as application servers

Chapter 4: Load Balancing

4.1 Definition

A load balancer is a component that sits in front of multiple server instances and distributes incoming traffic across them, based on a chosen algorithm, while continuously monitoring instance health.

4.2 Why it exists / problem it solves

Once you horizontally scale (Chapter 3), you have N servers instead of 1 — but a client can't be expected to know which of the N is available or least busy. The load balancer is the abstraction that lets clients talk to “the service” as a single address while the actual routing to a specific healthy instance happens transparently.

4.3 Internal working

1. Client sends a request to the load balancer's public address.
2. The load balancer consults its pool of registered, healthy backend instances.
3. It selects one instance based on the configured algorithm.
4. It forwards the request (proxying) or redirects the connection (depending on L4 vs L7, below).
5. In the background, independent of user traffic, it periodically sends health-check pings to every registered instance and removes any that fail to respond within a threshold.

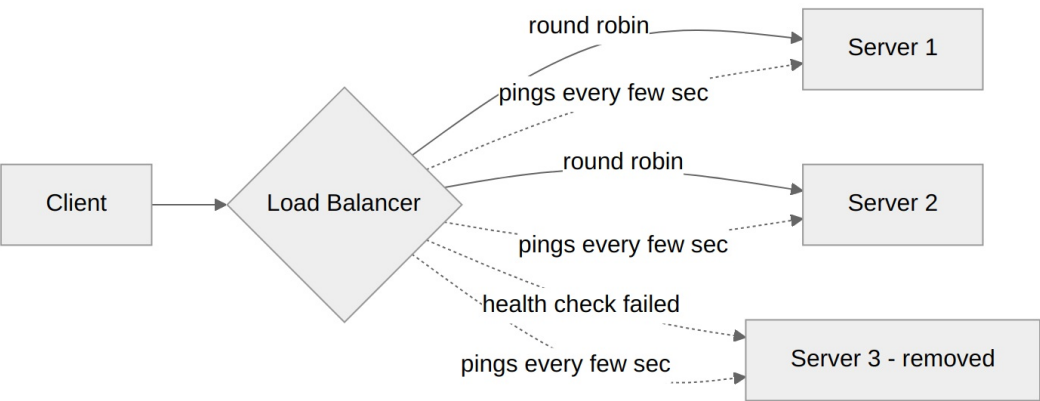


Diagram 3

4.4 L4 vs L7 Load Balancing

	Layer 4 (Transport)	Layer 7 (Application)
Operates on	Raw TCP/UDP connections	HTTP requests (headers, paths, cookies)
Routing decision	Based on IP + port only	Can route by URL path, header, cookie
Speed	Faster (less inspection overhead)	Slightly slower (parses full request)
SSL termination	Usually passthrough	Can terminate SSL at the LB
Example	AWS NLB	AWS ALB, NGINX, HAProxy (L7 mode)
Use case	Raw throughput, non-HTTP protocols	Microservices routing, content-based routing

4.5 Algorithms

Algorithm	How it decides	Best for
Round robin	Rotates through servers in order	Equal-capacity servers, uniform request cost
Weighted round robin	Rotates, but stronger servers get proportionally more	Mixed-capacity server fleet
Least connections	Sends to the server with fewest active connections	Requests with variable duration (uploads, streaming)
Least response time	Sends to the server with lowest latency + fewest connections	Latency-sensitive systems
IP hash / consistent hashing	Same client always maps to the same server	Session stickiness without a shared session store
Random / adaptive	Random pick, or dynamically weighted by live server metrics	Very large fleets where full state tracking is expensive

Note: Consistent hashing deserves its own deep dive — it’s not just a load balancing algorithm, it’s the foundational technique behind distributed caches, DynamoDB, and CDN routing. Covered fully in Chapter 14.

4.6 Session persistence (sticky sessions)

If your app tier isn’t fully stateless (Chapter 1), you sometimes need the same client to always reach the same server — e.g. an in-memory shopping cart. This is achieved via IP hash routing or a cookie the load balancer sets.

Warning: Sticky sessions reintroduce the exact single-point-of-failure problem horizontal scaling was meant to solve — if that one server dies, that client’s session dies with it. The better fix is almost always to externalize the state (Redis) and keep the app tier stateless, rather than relying on stickiness.

4.7 SSL Termination

Handling HTTPS decryption at the load balancer instead of at each backend server. This centralizes certificate management and offloads CPU-expensive decryption work from your application servers, at the cost of the traffic between LB and backend being unencrypted (usually acceptable within a private VPC).

4.8 Real production examples

- **NGINX / HAProxy** — self-hosted, extremely common for L7 load balancing in front of microservices
- **AWS ALB (Application Load Balancer)** — L7, HTTP-aware, supports path-based routing to different target groups
- **AWS NLB (Network Load Balancer)** — L4, extreme throughput, used for non-HTTP or ultra-low-latency traffic
- **Cloudflare Load Balancer** — operates at the edge, adds geographic routing on top of standard algorithms

Chapter 4 Summary

- Load balancer = traffic distribution + continuous health checking
- L4 = fast, protocol-agnostic. L7 = smarter, HTTP-aware, can route by content
- Algorithm choice depends on request uniformity (round robin) vs request variability (least connections)
- Sticky sessions are a workaround for statefulness, not a substitute for fixing it

Interview cheat sheet

- Always mention health checks — many candidates describe only the routing, not the failure detection
- Justify your algorithm choice against the specific request pattern in the problem, don’t default to round robin without reasoning
- Know the L4/L7 distinction cold — it’s a very common follow-up question

Common mistakes

- Confusing a load balancer with an API gateway (Chapter 5 draws this line precisely)
 - Not mentioning what happens when a server fails a health check mid-request
-

Chapter 5: API Gateway

5.1 Definition

An API gateway is a single entry point that sits in front of multiple, distinct backend services (often microservices), handling cross-cutting concerns — authentication, rate limiting, routing, transformation, aggregation — before forwarding requests to the appropriate service.

5.2 Difference from a load balancer

This is one of the most commonly confused pairs in system design interviews, so it’s worth being precise:

	Load Balancer	API Gateway
Fronts	Multiple <i>identical</i> instances of one service	Multiple <i>different</i> services
Core question answered	“Which instance?”	“Which service, and are you allowed?”
Auth-aware	No	Yes — validates tokens/API keys
Request transformation	No	Yes — can rewrite, aggregate, version
Protocol depth	Often L4, protocol-agnostic	Always L7, deeply HTTP/REST/GraphQL aware

In a real deployment, a request typically passes through **both**: the gateway first (deciding which microservice this belongs to, and authenticating it), then that microservice’s own load balancer (deciding which instance handles it).

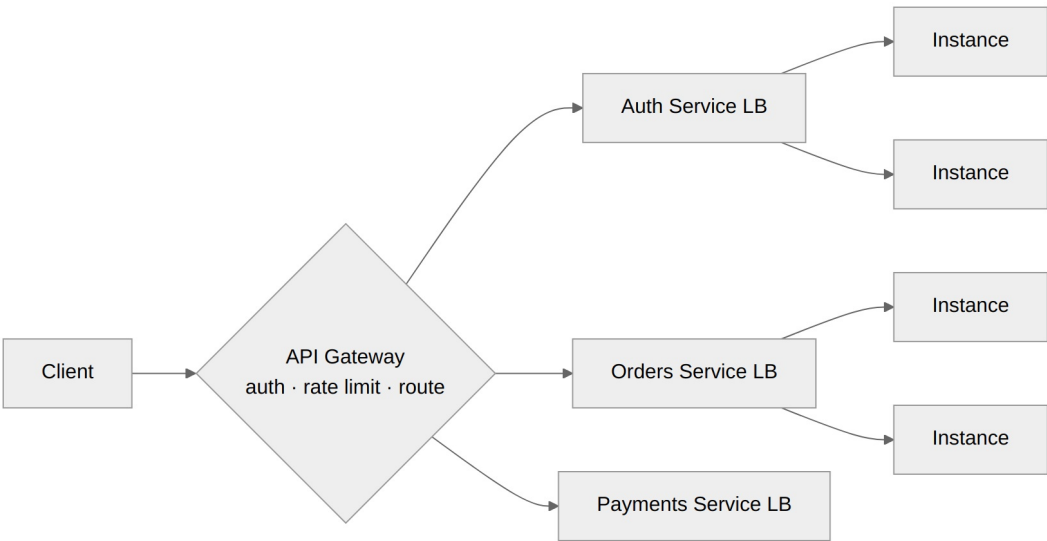


Diagram 4

5.3 Core responsibilities

- **Authentication** — validates identity (is this a real, logged-in user?)
- **Authorization** — validates permission (is this user *allowed* to do this?)
- **Routing** — maps URL paths to the correct backend service
- **Request transformation** — rewrites headers, reshapes payloads between client-facing and internal service formats
- **Aggregation** — combines responses from multiple services into a single response (avoids the client making 5 separate calls)
- **Rate limiting** — enforces per-client request limits (Chapter 6, in depth)
- **Monitoring & logging** — centralized visibility into every request crossing the system boundary

- **Circuit breaking** — stops forwarding requests to a service that's failing repeatedly, to avoid cascading failure (covered further in Chapter 18)

5.4 Authentication deep dive: JWT vs OAuth

- **JWT (JSON Web Token)** — a self-contained, signed token carrying claims (user ID, roles, expiry). The gateway verifies the signature without needing to call another service — fast, stateless, but revocation is hard (a JWT is valid until it expires, full stop, unless you add a blocklist).
- **OAuth 2.0** — a delegation *protocol*, not a token format. Lets a user grant a third-party app limited access to their data on another service (e.g. "sign in with Google") without sharing their password. Commonly issues JWTs as the actual access token.

5.5 When you *don't* need a gateway

If you have a single service (not a collection of microservices), a load balancer plus a thin auth middleware layer is sufficient — introducing a full API gateway adds an operational component and a network hop for no real benefit. The gateway earns its place specifically when you have multiple distinct services that need centralized auth, rate limiting, and routing.

5.6 Real production examples

- **Kong, AWS API Gateway, Apigee** — dedicated gateway products
- **Envoy** — often used as the data plane inside a service mesh, doing gateway-like duties at each service boundary
- **Spring Cloud Gateway** — common in Java/Spring microservice stacks

Chapter 5 Summary

- Gateway = smart front door for many services. Load balancer = traffic splitter for many copies of one service.
- A request typically passes through the gateway, then a per-service load balancer
- Don't introduce a gateway for a single-service system — it's solving a problem you don't have yet

Interview cheat sheet

- If asked "load balancer vs API gateway," lead with: "load balancer picks which instance, gateway picks which service and whether the request is even allowed"
- Mention circuit breaking as a gateway responsibility — it's a strong signal of depth
- Justify *when* you'd introduce a gateway, not just what it does

Common mistakes

- Using "API gateway" and "load balancer" interchangeably in an interview — the interviewer will probe this immediately
 - Forgetting authorization (permission) is distinct from authentication (identity)
-

Chapter 6: Rate Limiting

6.1 Definition

Rate limiting is the practice of controlling the number of requests a client can make to a system within a defined time window, rejecting or delaying requests that exceed the limit.

6.2 Why it exists / problem it solves

Without rate limiting, a single client — malicious or simply buggy (an infinite retry loop is a very common real-world cause) — can consume enough capacity to degrade service for every other client, or run up enormous cost on metered downstream resources (LLM API calls being a very direct example for AI-heavy systems).

6.3 Algorithms

Algorithm	Mechanism	Strength	Weakness
Fixed window counter	Count requests in a fixed clock-aligned window (e.g. per-minute)	Simple, cheap	Allows 2x burst at window boundaries
Sliding window log	Store exact timestamp of every request, count how many fall in the trailing window	Very accurate	Memory-expensive at scale
Sliding window counter	Weighted average of current + previous fixed window	Good accuracy, cheaper than log	Approximation, not exact
Token bucket	Tokens refill at a steady rate into a capped bucket; each request spends one	Allows controlled bursts, smooth long-term rate	Slightly more complex to implement
Leaky bucket	Requests queue and drain at a constant output rate	Perfectly smooths bursts	Adds latency; queue can overflow under sustained load

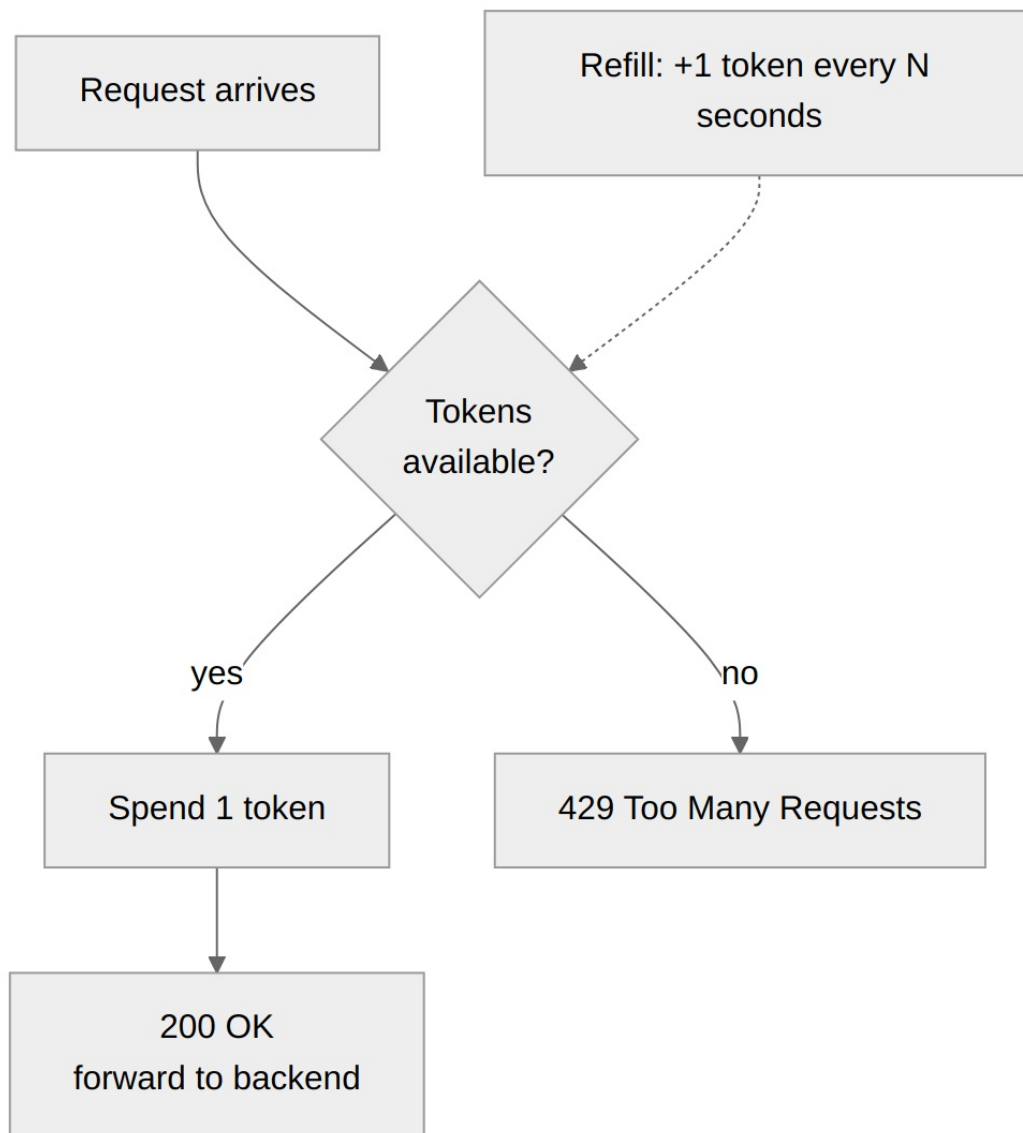


Diagram 5

6.4 Redis implementation (token bucket, simplified)

```
import time

def is_allowed(redis, key: str, capacity: int, refill_rate: float) -> bool:
    now = time.time()
    bucket = redis.hgetall(key)
    tokens = float(bucket.get(b"tokens", capacity))
    last_refill = float(bucket.get(b"last_refill", now))

    # Refill based on elapsed time
    elapsed = now - last_refill
    tokens = min(capacity, tokens + elapsed * refill_rate)

    if tokens < 1:
        redis.hset(key, mapping={"tokens": tokens, "last_refill": now})
        return False

    tokens -= 1
    redis.hset(key, mapping={"tokens": tokens, "last_refill": now})
    return True
```

Tip: In production, wrap this logic in a Redis Lua script (`EVAL`) so the read-modify-write is atomic. Without it, two concurrent requests can both read the same token count before either writes back — a classic race condition that silently lets more requests through than intended.

6.5 Distributed rate limiting

A single Redis instance works when one gateway node enforces the limit. At larger scale, you either: - Centralize all rate-limit checks through a shared Redis cluster (simple, but Redis becomes a dependency every request touches) - Approximate with local counters per node plus periodic synchronization (faster, but temporarily allows some overshoot)

6.6 Where it lives in the request path

Rate limiting should happen as early and cheaply as possible — ideally at the API gateway or edge, *before* the load balancer even routes the request. There's no reason to spend load-balancing and backend compute on a request you're about to reject.

6.7 Real production examples

Stripe, GitHub, and most public APIs use token-bucket-style limiting and communicate the state via response headers (`X-RateLimit-Remaining` , `Retry-After`) so well-behaved clients can self-throttle before hitting the wall.

Chapter 6 Summary

- Rate limiting protects backend capacity, enforces fairness, and controls cost on metered downstream calls
- Token bucket is the production default — burst-tolerant with a strict long-term average
- Enforce as early in the request path as possible; make the check atomic in a distributed environment

Interview cheat sheet

- Token bucket is almost always the “correct” answer to “how would you rate limit this API” unless the interviewer specifically wants smoothing (then leaky bucket)
- Mention the race condition risk and atomic Lua script fix — this is a strong differentiator
- Rate limit by API key/user ID, not IP, when clients may share a NAT/proxy

Common mistakes

- Rate limiting only at the app server instead of at the gateway/edge — wastes compute on rejected requests
 - Forgetting to make the check atomic under concurrency
-

Chapter 7: Middleware

7.1 Definition

Middleware is a chain of functions that a request passes through — in order — between arriving at the server and reaching the actual business-logic handler, and again (in reverse) on the way back out as a response. Each layer can inspect, modify, reject, or pass along the request/response.

7.2 The onion model

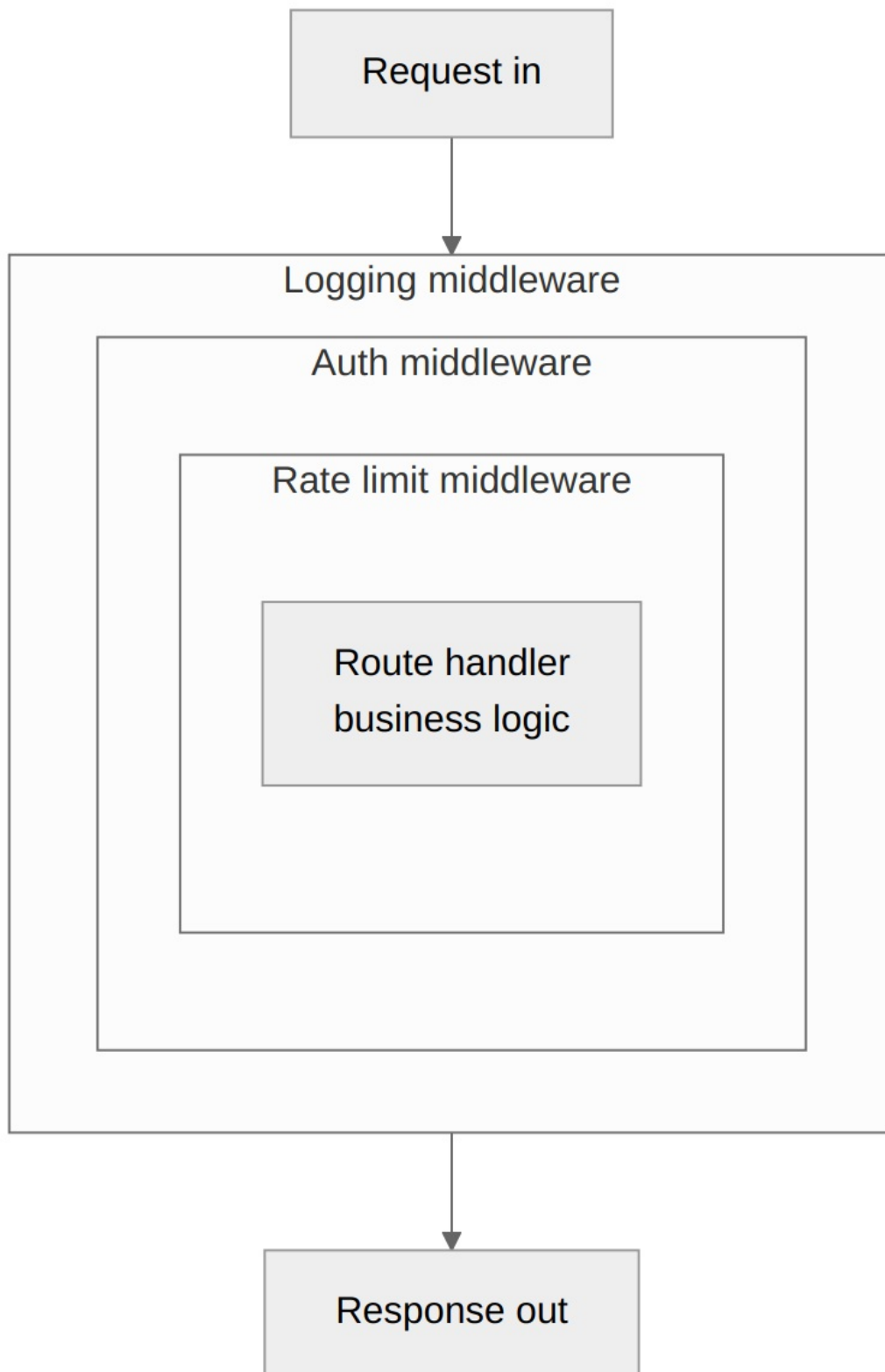


Diagram 6

The request flows *inward* through each layer to reach the handler, then the response flows *outward* through the same layers in reverse — each layer gets a second chance to act, this time on the response (e.g. logging the final status code and duration).

7.3 Why it exists / problem it solves

Without middleware, every route handler would need to duplicate auth checks, logging, and error handling internally — copy-pasted across every endpoint. Middleware applies cross-cutting logic once, centrally, to every request that passes through the chain — a direct application of the DRY principle to concerns that aren't specific to any one route.

7.4 Common middleware types

Type	Responsibility
Logging	Record method, path, status, duration for every request
Authentication	Verify identity; short-circuits with 401 if invalid
Rate limiting	Enforce per-client request caps (Chapter 6)
CORS	Control which frontend origins may call the API from a browser
Error handling	Catch unhandled exceptions anywhere downstream, return clean error responses
Compression	Gzip response bodies before sending
Body parsing	Convert raw request bytes into a usable object (e.g. JSON → dict)

7.5 Execution order matters

Middleware is registered in a specific order, and that order determines behavior. Consider:

```
app.add_middleware(LoggingMiddleware)
app.add_middleware(RateLimitMiddleware)
app.add_middleware(AuthMiddleware)

@app.post("/generate-video")
async def generate_video(request):
    ...
```

If `AuthMiddleware` rejects the request, `generate_video` is never invoked — the short-circuit happens before any business logic runs, which is exactly why rejection via middleware is cheap. But note logging still wraps everything outermost here, so even rejected requests get logged — a deliberate ordering choice, not an accident.

Warning: Placing rate limiting *after* authentication means you're spending compute authenticating a request you might reject anyway for exceeding rate limits. Placing it *before* auth means unauthenticated clients can still exhaust the rate limit bucket meant for legitimate users. There's no universally correct order — it's a trade-off you should be able to justify explicitly.

7.6 Framework implementations

Framework	Middleware mechanism
FastAPI	<code>@app.middleware("http")</code> decorator or <code>BaseHTTPMiddleware</code> class; async-native
Express (Node.js)	<code>app.use(fn)</code> — the original popularizer of this pattern
Spring Boot	<code>HandlerInterceptor</code> / <code>Filter</code> chain
ASP.NET Core	<code>IMiddleware</code> pipeline configured in <code>Program.cs</code>

7.7 Context propagation and dependency injection

A common production need: middleware needs to pass data forward to the route handler (e.g. the authenticated user object, extracted from the token). In FastAPI this is typically done via `request.state`; in async frameworks generally, be careful — data attached to a shared context can leak across concurrent requests if not scoped correctly per-

request. This is a frequent source of subtle bugs in async Python specifically, and is covered in depth alongside `async/await` in Chapter 16.

7.8 Performance impact

Every middleware layer adds a small amount of latency (function call overhead, plus whatever work it does — a regex check, a Redis round-trip for rate limiting, a JWT signature verification). This is usually negligible individually, but a long chain of poorly optimized middleware (e.g. a synchronous blocking call inside an async middleware) can meaningfully hurt p99 latency under load. Keep the chain lean, and never do blocking I/O inside middleware without awaiting it properly.

Chapter 7 Summary

- Middleware = ordered chain of functions wrapping the route handler, applied to every request (the “onion model”)
- Exists to centralize cross-cutting concerns (auth, logging, rate limiting) instead of duplicating them per-route
- Order is a deliberate architectural decision with real trade-offs, not an arbitrary registration sequence
- Watch for context-leakage bugs in async frameworks specifically

Interview cheat sheet

- Be ready to justify middleware ordering, not just list what middleware you’d add
- Mention the short-circuit behavior explicitly — it’s why rejection via middleware is cheap
- If asked about FastAPI specifically, mention `request.state` for context propagation and the async context-leak risk

Common mistakes

- Doing expensive synchronous work inside middleware without considering its effect on every single request’s latency
 - Registering auth after rate limiting (or vice versa) without a deliberate reason
-

Chapter 8: CAP Theorem

8.1 Definition

In a distributed system, when a network partition occurs, you can only guarantee two of the following three properties simultaneously:

- **Consistency (C)** — every read receives the most recent write, or an error
- **Availability (A)** — every request receives a non-error response (not guaranteed to be the latest data)
- **Partition Tolerance (P)** — the system keeps operating despite network partitions between nodes

8.2 Why it exists / the real choice

Formalized by Eric Brewer in 2000 and formally proven by Gilbert and Lynch in 2002. Here's the part most summaries skip: partition tolerance isn't really optional. In any distributed system with nodes on separate machines, the network *will* fail sometimes — cables get cut, packets get dropped, a data center loses connectivity. So the real, practical choice CAP forces on you is: **when a partition happens, do you sacrifice Consistency or Availability?**

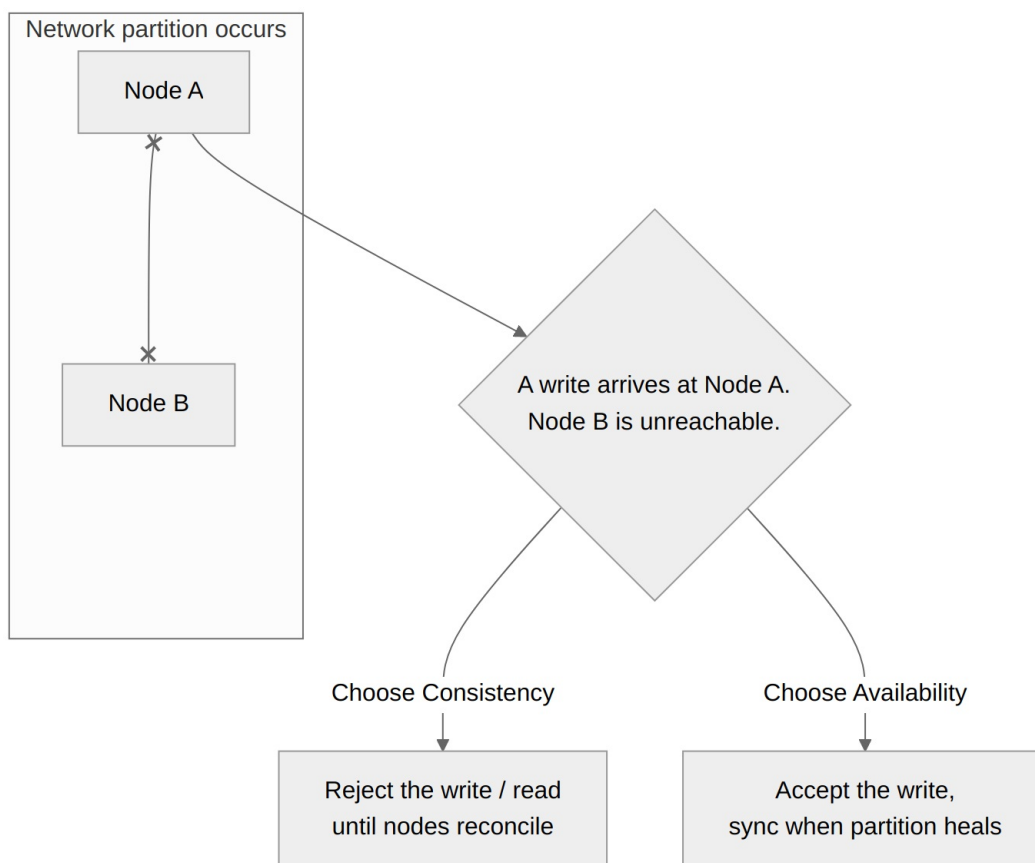


Diagram 7

8.3 PACELC — the extension people actually need

CAP only describes behavior *during* a partition. But even when the network is perfectly healthy, there's still a trade-off between latency and consistency (waiting for all replicas to confirm a write takes longer than trusting one). PACELC states it fully:

If Partitioned, choose **Availability** or **Consistency** — **Else** (normal operation), choose **Latency** or **Consistency**.

This is why “AP vs CP” alone is an incomplete way to describe a system — you also need to know its behavior when

nothing is even broken.

8.4 Real systems mapped to CAP/PACELC

System	Primary choice	Notes
Cassandra	AP (tunable)	Quorum reads/writes let you dial consistency per-query
MongoDB	CP by default	Single primary per replica set; can be tuned toward availability
PostgreSQL	CP	Classically single-node; not natively partition-tolerant without extensions
Redis (Cluster)	Leans AP	Async replication between nodes can lose recent writes on failover
CockroachDB	CP	Distributed SQL using Raft consensus for strong consistency
Google Spanner	CP, near-CA in practice	Uses atomic-clock-synchronized “TrueTime” to make CP behave close to always-available — a famous, expensive engineering workaround, not a violation of the theorem
DynamoDB	AP by default	Offers optional strongly-consistent reads at a latency/availability cost

Chapter 8 Summary

- Partition tolerance is mandatory in practice — the real decision is Consistency vs Availability during a partition
- PACELC extends this to normal operation: Latency vs Consistency
- No system “beats” CAP — Spanner’s near-always-available CP behavior comes from enormous engineering investment (custom hardware, atomic clocks), not a loophole

Interview cheat sheet: Never say “CA systems exist” without qualifying it — in a true distributed, multi-node system, partition tolerance can’t be opted out of. When asked to design something, state your CAP position explicitly and justify it against the read/write pattern (Chapter 2).

Common mistake: Treating CAP as a one-time global choice for an entire company’s stack, rather than a per-system (sometimes per-operation) decision — e.g. Amazon famously runs both AP systems (shopping cart, DynamoDB) and CP systems (payments) side by side, on purpose.

Chapter 9: Caching

9.1 Definition

Caching stores a copy of frequently accessed data in a faster-access layer, so future requests can be served without repeating expensive work (a database query, an API call, a computation).

9.2 The cache hierarchy

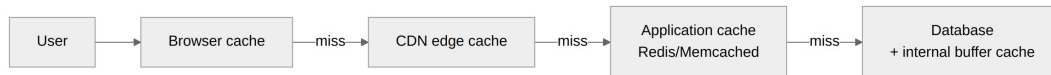


Diagram 8

Each layer exists to catch what the layer before it missed. The closer to the user a cache hit happens, the cheaper and faster it is — a browser cache hit costs nothing on your infrastructure at all.

9.3 Caching strategies

Strategy	How it works	Trade-off
Cache-aside (lazy loading)	App checks cache; on miss, reads DB and populates cache itself	Most common pattern; first request after expiry is always slow
Read-through	Cache library handles the DB read transparently; app only ever talks to the cache	Simpler app code; couples cache library to DB access
Write-through	Write goes to cache and DB synchronously, in the same operation	Cache always consistent; write latency = cache + DB write
Write-back (write-behind)	Write goes to cache immediately; DB updated asynchronously later	Fastest writes; risk of data loss if cache fails before flush
Write-around	Write goes directly to DB, bypassing cache entirely	Good when written data isn't re-read soon; first read is a guaranteed miss

9.4 Invalidation and eviction

As the old joke goes: there are only two hard problems in computer science — cache invalidation, naming things, and off-by-one errors.

Invalidation approaches: TTL expiration (simplest, but data can be stale for up to the TTL window), explicit invalidation on write (accurate, but requires discipline to never miss a code path), versioned cache keys (sidesteps invalidation — old versions just age out naturally).

Eviction policies (when the cache is full and needs to make room):

Policy	Evicts	Best for
LRU (Least Recently Used)	Item not accessed in the longest time	General-purpose default
LFU (Least Frequently Used)	Item accessed the fewest times	Workloads with a stable “hot set”
FIFO	Oldest inserted item, regardless of usage	Simple, rarely optimal
Random	Arbitrary item	Surprisingly competitive at very large scale; near-zero bookkeeping cost

9.5 The four cache pathologies you'll be asked about

- **Cache stampede (thundering herd)** — a hot key expires, and many concurrent requests miss simultaneously, all hammering the DB at once. *Fix*: a lock/mutex so only one request repopulates the cache while others wait, or probabilistic early expiration.
- **Cache penetration** — repeated requests for a key that doesn't exist in cache *or* the database, so every request pointlessly bypasses the cache. *Fix*: cache the “not found” result briefly, or check existence cheaply with a Bloom filter (Chapter 18) before ever hitting the DB.
- **Cache avalanche** — many keys expire at the same moment (e.g. all set with the same TTL at the same time), causing a simultaneous burst of DB load. *Fix*: add random jitter to TTLs so expirations spread out.
- **Hot keys** — one key (a viral post, a trending product) receives wildly disproportionate traffic, overloading the single cache node/shard responsible for it. *Fix*: replicate that specific key across multiple nodes, or add a small local in-process cache in front of the distributed cache.

Chapter 9 Summary

- Caching sits in layers — browser, CDN, application, database — each catching the previous layer's misses
- Cache-aside is the default pattern; write-through/write-back trade write latency against durability risk
- The four pathologies (stampede, penetration, avalanche, hot keys) are the actual interview differentiators — knowing the *names* of the strategies is table stakes, knowing these failure modes is what separates strong answers

Interview cheat sheet: When proposing a cache, immediately state your invalidation strategy — an unqualified “I’d add a cache” is an incomplete answer. Mention TTL jitter proactively; it signals you’ve actually operated a cache in production.

Common mistake: Proposing caching as a fix for a write-heavy workload — caching helps reads, not writes; a write-heavy bottleneck needs Chapter 10's tools instead.

Chapter 10: Database Scaling

10.1 Replication

Replication copies data from a **primary** (leader) node to one or more **replica** (follower) nodes. The primary handles all writes; replicas can serve reads, spreading read load and providing redundancy.

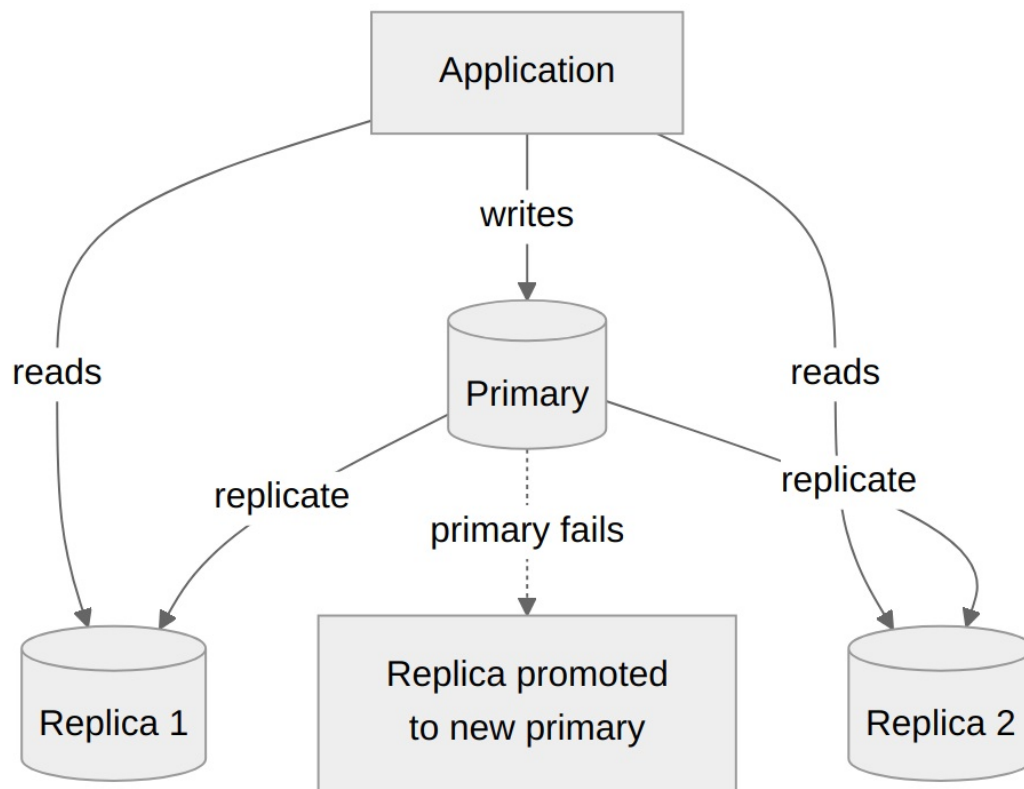


Diagram 9

Sync vs async replication: synchronous replication waits for replicas to confirm before acknowledging the write (safer, higher latency); asynchronous replication acknowledges immediately and replicates in the background (faster, risk of losing the most recent writes if the primary fails before replicating).

Failover: if the primary dies, a replica must be promoted. This can be manual (safer, slower) or automatic via a consensus protocol like Raft (faster recovery, more moving parts to get right).

10.2 Sharding (horizontal partitioning)

Replication solves *read* scaling and availability — it does nothing for *write* scaling or total data volume, since every replica still holds a full copy of everything. Sharding splits the data itself across multiple independent database instances, each holding only a slice.

Shard key — the field used to decide which shard a row belongs to (e.g. `user_id`, or `hash(user_id) % N`). Choosing this key well is the single most consequential decision in a sharded system.

Problem	Cause	Mitigation
Hot partitions	Shard key distributes data unevenly (e.g. sharding by signup date puts all recent activity on one shard)	Choose a high-cardinality, evenly-distributed key; use consistent hashing (Chapter 14)
Cross-shard joins	Related data lives on different shards; a relational JOIN can't span them	Denormalize, or resolve joins at the application layer
Rebalancing	Adding/removing a shard requires moving data	Consistent hashing minimizes the fraction of data that must move

10.3 Distributed SQL

Systems like **CockroachDB**, **Google Spanner**, and **YugabyteDB** combine sharding + replication + distributed consensus to give you horizontal scalability *with* SQL semantics and strong consistency — directly solving the traditional “SQL doesn’t scale horizontally” problem, at the cost of significantly more operational complexity than a single Postgres instance.

Chapter 10 Summary

- Replication scales reads and adds availability; it does not scale writes or reduce total storage per node
- Sharding scales writes and total data volume, at the cost of losing easy cross-shard joins and transactions
- Shard key choice is the highest-leverage decision in a sharded system — get it wrong and you’ve just moved the hotspot problem, not solved it

Interview cheat sheet: Always propose replication *before* sharding — sharding is a bigger, harder step that should be justified by an actual write-scaling need, not reached for by default.

Common mistake: Sharding by a low-cardinality or sequential key (timestamp, auto-increment ID) — this reliably produces hot partitions because recent/related data clusters on one shard.

Chapter 11: SQL vs NoSQL

11.1 ACID vs BASE

	ACID (traditional SQL)	BASE (typical NoSQL)
Meaning	Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventually consistent
Philosophy	Correctness above all, even under failure	Availability above all; consistency catches up
Best for	Financial transactions, inventory, anything where partial writes are unacceptable	High-scale reads/writes where brief staleness is tolerable

11.2 Normalization vs Denormalization

Normalization (typical SQL default) minimizes data duplication by splitting data into related tables, joined at query time — reduces storage and update anomalies, at the cost of join overhead. **Denormalization** (common in NoSQL, and sometimes deliberately in SQL for read-heavy paths) duplicates data across documents/rows to avoid joins entirely, trading storage and write complexity for much faster reads.

11.3 NoSQL categories

Type	Model	Example	Best for
Document	Flexible, nested JSON-like documents	MongoDB, Firestore	Catalogs, content with varying shape
Column-family	Wide-column, optimized for write-heavy, time-ordered data	Cassandra, HBase	Time-series, activity logs, IoT
Key-value	Simplest possible model — just get/set by key	Redis, DynamoDB	Caching, session storage, feature flags
Graph	Relationships as first-class citizens	Neo4j	Social graphs, recommendation engines, fraud detection

11.4 Indexes, briefly

A database index (typically a B-tree) trades write cost and storage for dramatically faster reads on the indexed column — every write must also update the index. Over-indexing is a real anti-pattern: each additional index slows every write to that table.

11.5 When to choose which

Choose SQL when: the data is inherently relational, transactional integrity matters (money, inventory, bookings), and the schema is stable.

Choose NoSQL when: you need horizontal write scale beyond what a single relational primary can give you, the schema needs to evolve frequently, or the access pattern is naturally document/graph/key-value shaped rather than relational.

Note: Most large production systems use both — a relational database for the transactional core (orders, billing) and one or more NoSQL stores for specific high-scale needs (session cache, activity feed, search index). “SQL vs NoSQL” is rarely an either/or at the company level.

Chapter 11 Summary

- ACID prioritizes correctness; BASE prioritizes availability and scale, accepting eventual consistency
- Denormalization trades storage/write complexity for read speed — the opposite trade-off of classic normalization
- The right answer is almost always “the right tool per workload,” not a single company-wide choice

Interview cheat sheet: Justify your database choice against the specific access pattern in the prompt (read:write ratio, relational vs flat, consistency needs) — never say “I’d use NoSQL because it scales better” without that justification; it’s not universally true.

Common mistake: Assuming NoSQL means “no schema at all” — most NoSQL databases still benefit heavily from a consistent, planned document/row shape; “schemaless” just means the *database* doesn’t enforce it, the application still needs to.

Chapter 12: Message Queues

12.1 Definition and need

A message queue decouples the producer of work from the consumer of it — the producer publishes a message and moves on; the consumer processes it whenever it's ready. This absorbs traffic spikes, enables async processing, and lets producer and consumer scale, fail, and deploy independently.

12.2 Core concepts

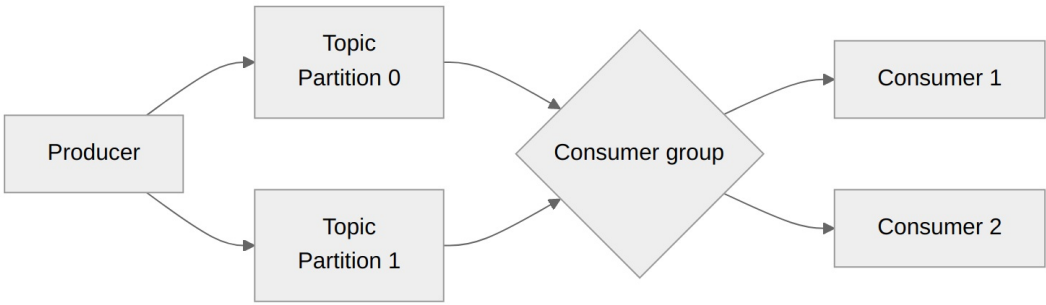


Diagram 10

- **Producer** — publishes messages
- **Broker** — the queue/messaging system itself (Kafka, RabbitMQ)
- **Topic** — a named stream of messages
- **Partition** — a topic is split into partitions for parallelism; order is guaranteed *within* a partition, not across the whole topic
- **Offset** — a consumer's position/bookmark within a partition
- **Consumer group** — a set of consumers sharing the work of a topic; each partition is consumed by exactly one member of the group at a time

12.3 Delivery guarantees

Guarantee	Behavior	Risk
At-most-once	Fire and forget, no retry	Messages can be silently lost
At-least-once	Retries on failure/timeout	Messages can be delivered more than once — requires idempotent consumers
Exactly-once	Combines idempotency + transactional offset commits	Hardest and most expensive to guarantee correctly

Tip: In practice, most systems use at-least-once delivery and achieve *effectively* exactly-once behavior by making the consumer idempotent (Chapter 18) rather than trying to achieve true exactly-once at the broker level.

12.4 Retries and Dead Letter Queues

A message that fails processing is retried, typically with exponential backoff (Chapter 18). After N failed attempts, it's routed to a **Dead Letter Queue (DLQ)** — a holding area for messages that couldn't be processed, so they don't block the rest of the queue indefinitely and can be inspected manually.

12.5 Backpressure

If consumers can't keep up with producers, the queue absorbs the growing backlog rather than crashing anything

downstream — this is one of the queue’s core benefits. But an unbounded backlog just delays the problem; production systems monitor queue depth and add consumer capacity (or apply backpressure upstream to producers) before it grows without bound.

12.6 Kafka vs RabbitMQ vs SQS vs Pub/Sub

	Kafka	RabbitMQ	AWS SQS	Google Pub/Sub
Model	Distributed log	Traditional message broker	Managed queue	Managed pub/sub
Ordering	Per-partition	Per-queue (with care)	FIFO queues available	Per-ordering-key
Throughput	Very high	Moderate	Moderate, fully managed	High, fully managed
Message retention	Configurable, can replay old messages	Consumed messages are gone	Short retention window	Configurable
Best for	High-throughput event streaming, event sourcing	Complex routing, task queues	Simple managed queueing on AWS	Managed pub/sub on GCP

Chapter 12 Summary

- Queues decouple producer and consumer, absorbing bursts and enabling independent scaling
- Ordering is only guaranteed within a partition — design your partition key around what needs ordering
- At-least-once + idempotent consumers is the practical default; true exactly-once is rarely worth the complexity

Interview cheat sheet: If asked “how do you prevent duplicate processing,” the answer is idempotency (a dedup key on the consumer side), not “use exactly-once delivery” — the latter is a much heavier and often unnecessary guarantee.

Common mistake: Assuming global ordering across an entire topic — it’s only guaranteed within a single partition, and most beginners miss this until a bug forces the lesson.

Chapter 13: Monolith vs Microservices

13.1 Monolith

A single deployable unit containing all application logic.

Pros: simple to develop and deploy early on, no network overhead between modules (everything is an in-process function call), transactions are trivially ACID (one shared database).

Cons: scaling requires scaling the entire application even if only one part is under load, a bug anywhere has a large blast radius, deploys get slower and riskier as the codebase grows, and large teams working in the same codebase step on each other.

13.2 Microservices

Independently deployable services, each typically owning its own data store.

Pros: independent scaling per service, independent deployment (teams ship on their own schedule), fault isolation (one service failing doesn't necessarily take down others), team autonomy.

Cons: real operational complexity — service discovery, network reliability between services, distributed tracing just to debug a single user request, and the hard problem below: distributed transactions.

13.3 Migration strategy: the Strangler Fig pattern

Rather than a risky big-bang rewrite, incrementally route specific routes or features to new microservices while the monolith continues serving everything else — over time, the monolith shrinks until it's fully replaced (or reduced to a small remaining core). Named after the strangler fig vine, which gradually envelops and eventually replaces its host tree.

13.4 Service discovery

In a microservices system, instances come and go dynamically (scaling, deploys, failures) — hardcoding IP addresses doesn't work. Service discovery (Consul, Eureka, or Kubernetes' built-in DNS-based service objects) lets services find each other's current network location dynamically.

13.5 Distributed transactions and the Saga pattern

You can't wrap a transaction across multiple services' independent databases in a single ACID commit. The **Saga pattern** solves this by breaking a business transaction into a sequence of local transactions, each with a defined **compensating action** to undo it if a later step fails.

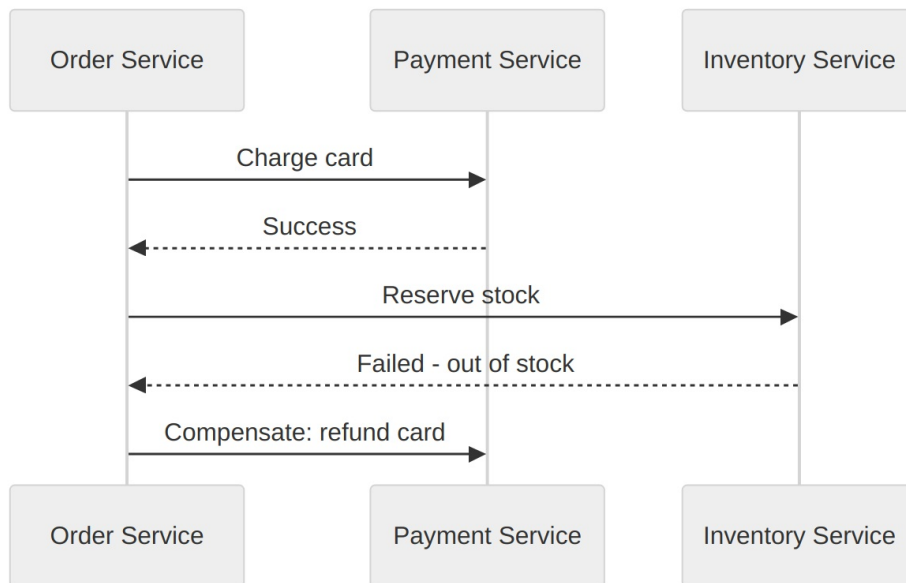


Diagram 11

Two coordination styles: **choreography** (each service reacts to events published by the previous one — no central coordinator, more decoupled, harder to trace) and **orchestration** (a central coordinator explicitly directs each step — easier to reason about, but the coordinator becomes a critical component).

13.6 Communication choices

	REST	gRPC	Message queue
Style	Synchronous HTTP/JSON	Synchronous, binary (Protobuf)	Asynchronous
Speed	Moderate	Fast (binary, HTTP/2)	N/A - decoupled
Best for	Public APIs, simplicity	Internal service-to-service, high throughput	Decoupled, event-driven flows

Chapter 13 Summary

- Start with a monolith unless you have a concrete reason not to — most systems don't need microservices' operational overhead from day one
- The Strangler Fig pattern is the safe, incremental path from monolith to microservices
- Distributed transactions require the Saga pattern's compensating actions since cross-service ACID isn't available

Interview cheat sheet: If asked "monolith or microservices" for a new system, a strong answer often starts with "monolith first, and here's the specific signal that would tell me to split it" — this shows judgment over reflexive trend-following.

Common mistake: Defaulting to microservices for a small team or early-stage product — the operational tax (Chapter 17's observability requirements alone) frequently outweighs the benefit before you actually have scaling or team-autonomy pain.

Chapter 14: Consistent Hashing

14.1 Definition

Consistent hashing maps both data (keys) and servers onto a fixed circular hash space (a “ring”), such that adding or removing a server only requires remapping a small fraction of keys — not nearly all of them.

14.2 The problem it solves

With naive `hash(key) % N` routing, changing N (adding or removing even a single server) reshuffles almost every key’s target server. For a distributed cache, that’s a mass cache-miss event; for a distributed database, that’s a mass, unnecessary data migration.

14.3 How the ring works

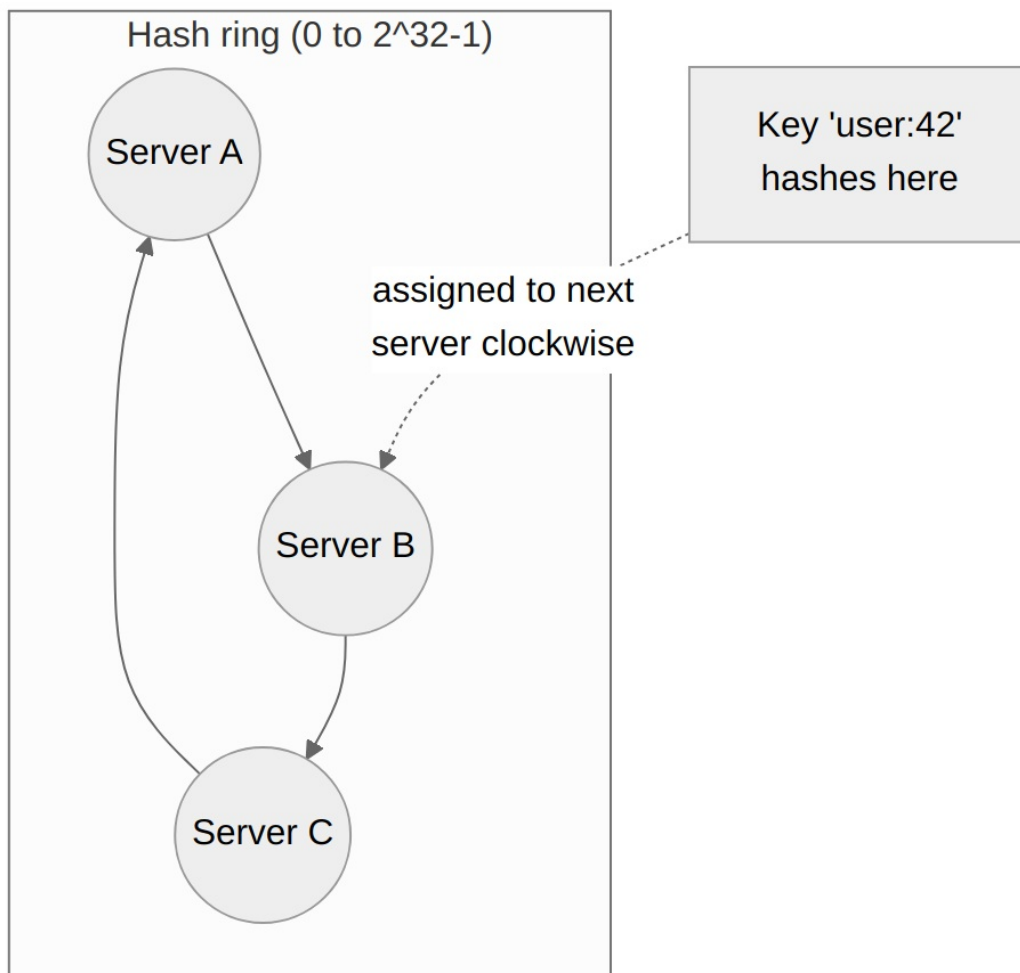


Diagram 12

Servers are hashed onto points on the ring. A key is assigned to the first server found going clockwise from the key’s own hash position. When a server is added, it only takes over the portion of keyspace between itself and its counter-clockwise neighbor — everything else on the ring is untouched.

14.4 Virtual nodes

A single physical server mapped to just one point on the ring gets an uneven, unpredictable slice of the keyspace.

The fix: map each physical server to **many** points on the ring (hundreds, via different hash seeds). This spreads load far more evenly, and when a real server is added or removed, the affected keyspace is small and evenly distributed rather than one large contiguous chunk.

14.5 Replication on the ring

To survive a node failure, a key is also stored on the next $R-1$ servers going clockwise from its primary position — this is exactly how Cassandra and DynamoDB achieve durable replication without a single leader coordinating every write.

14.6 Real usage

- **DynamoDB / Cassandra** — partition data across nodes using this exact mechanism
- **Redis Cluster** — uses a related but simpler scheme (16,384 fixed hash slots distributed across nodes) rather than a literal ring — worth knowing the distinction if asked directly
- **CDN request routing** — routes a request to the nearest/least-loaded edge server using similar principles

Chapter 14 Summary

- Consistent hashing minimizes data movement when the server pool changes, unlike `hash % N`
- Virtual nodes are what make load distribution actually even in practice — the ring alone isn't enough
- Replication on the ring (N servers clockwise) gives durability without a single point of coordination

Interview cheat sheet: If a design involves adding/removing nodes from a distributed cache or database, consistent hashing is almost always the expected answer for “how do you avoid remapping everything.”

Common mistake: Describing the ring without mentioning virtual nodes — a ring with one point per physical server still has real, painful load-distribution problems in practice.

Chapter 15: CDN Internals

15.1 Definition

A Content Delivery Network is a geographically distributed network of proxy (edge) servers that cache content close to end users, reducing latency and offloading traffic from the origin server.

15.2 Edge servers and POPs

Points of Presence (POPs) are physical data centers spread across the globe, each hosting cache servers. A user's request is routed to the nearest (or least-loaded) POP rather than all the way to the origin.

15.3 Routing to the nearest POP

- **Geo-aware DNS** — the DNS resolver returns a different IP address depending on the requester's geographic location
- **Anycast** — the *same* IP address is announced from multiple POPs simultaneously; internet routing (BGP) naturally sends the request to the topologically nearest announcing location

15.4 Cache hierarchy and flow

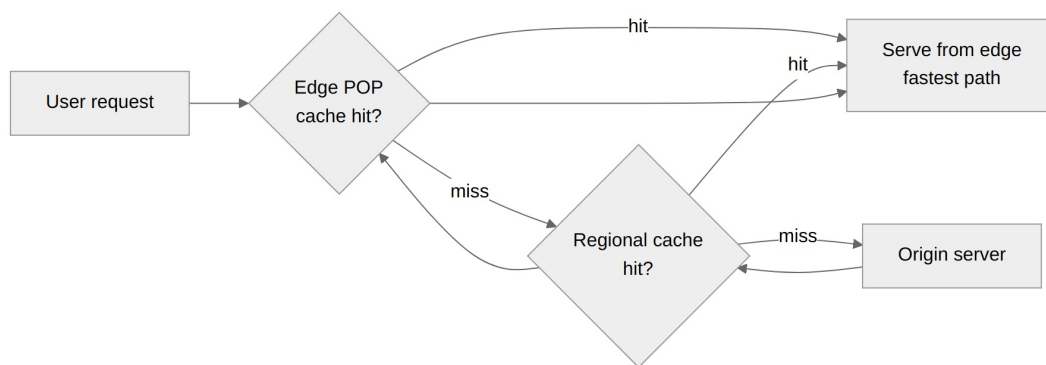


Diagram 13

A miss at the edge checks a regional cache tier before ever reaching the origin — this shields origin infrastructure from the vast majority of traffic even during a cold cache.

15.5 What gets cached, and how it's kept fresh

Static assets (images, CSS, JS, video segments) are cached by default. Dynamic content *can* be cached too, with careful `Cache-Control` headers, or handled with edge compute (Cloudflare Workers, Lambda@Edge) that runs logic at the POP itself.

Invalidation approaches, from least to most reliable in practice: `max-age` TTL expiration, explicit purge API calls, and **versioned URLs** (embedding a content hash in the filename) — the last one is the most robust in production because it sidesteps invalidation entirely; a new version is simply a new URL, and old cached versions just age out naturally.

15.6 Compression and optimization

Gzip/Brotli compression of text assets, and on-the-fly image transformation (format conversion to WebP/AVIF, responsive resizing) performed at the edge — reducing bytes transferred without any origin involvement.

15.7 Real production examples

Cloudflare, Fastly, AWS CloudFront — all combine the above with additional edge-compute and security capabilities (DDoS mitigation, WAF) layered on top of pure caching.

Chapter 15 Summary

- CDNs push content close to users via geographically distributed POPs, using anycast or geo-DNS to route requests to the nearest one
- A regional cache tier between edge and origin absorbs edge misses without hitting origin infrastructure
- Versioned URLs are the most reliable cache-freshness strategy — they avoid the invalidation problem instead of solving it

Interview cheat sheet: Any system serving large static assets (video, images) at scale should mention a CDN early — trying to serve that traffic from application servers is a common and costly design mistake to flag.

Common mistake: Forgetting that dynamic, personalized content generally *shouldn't* be cached at the CDN layer without careful per-user cache keys — caching one user's personalized page and serving it to another is a real, embarrassing production bug class.

Chapter 16: Async Programming

16.1 Blocking vs non-blocking I/O

A **blocking** call halts the executing thread until the operation completes (e.g. waiting on a database round-trip). A **non-blocking** call lets the thread do other work while waiting, and resumes handling the result once it's ready.

16.2 The event loop

A single-threaded loop that manages many concurrent I/O-bound operations by switching between them whenever one is waiting on I/O, instead of dedicating an entire thread to each. This is exactly why Node.js and Python's `asyncio` can juggle thousands of concurrent connections on a single thread — they're not doing parallel *computation*, they're eliminating wasted idle-wait time.

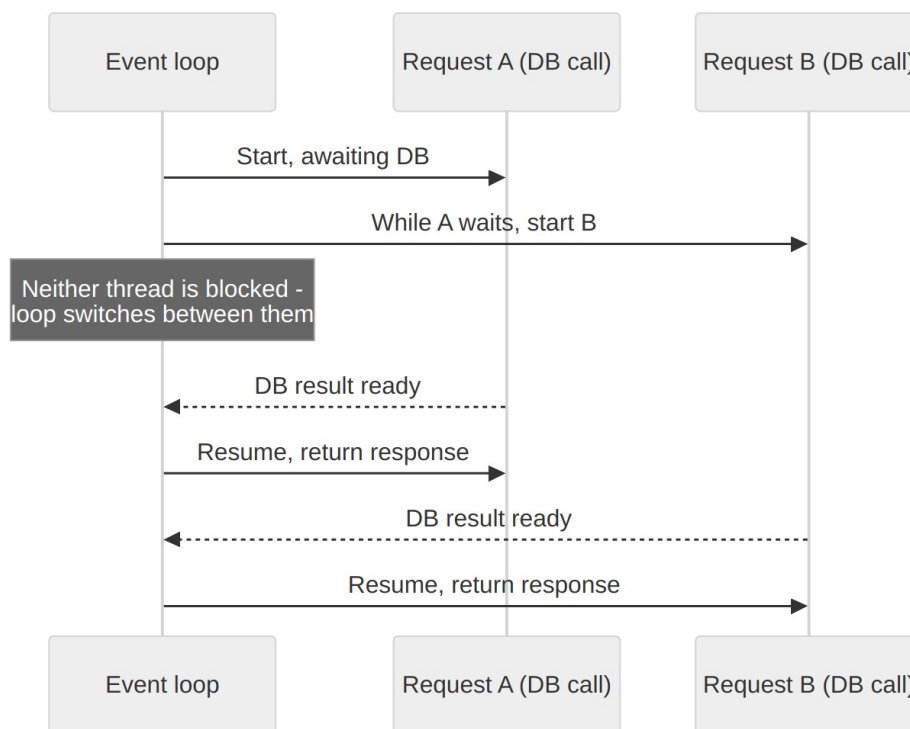


Diagram 14

16.3 Concurrency vs parallelism

Concurrency is handling multiple tasks by interleaving them (not necessarily at the same instant). **Parallelism** is literally executing multiple tasks at the same instant, requiring multiple CPU cores. `async / await` gives you concurrency for I/O-bound work — it does **not** give you parallelism for CPU-bound work, since a single event loop thread can still only run one piece of Python code at a time.

16.4 The most common production bug: blocking the event loop

```
# WRONG - blocks the entire event loop for every concurrent request
@app.get("/data")
async def get_data():
    result = requests.get("https://slow-api.com") # synchronous call!
    return result.json()

# RIGHT - non-blocking, lets other requests proceed while waiting
@app.get("/data")
async def get_data():
    async with httpx.AsyncClient() as client:
        result = await client.get("https://slow-api.com")
    return result.json()
```

A single synchronous, blocking call inside an `async def` handler freezes the *entire* event loop — meaning every other concurrent request on that server, not just this one, stalls until it completes. This is one of the most common and most damaging production bugs in async Python codebases.

16.5 Escaping blocking work: thread pools and process pools

For unavoidable blocking calls (a legacy synchronous library), offload to a **thread pool** (`run_in_executor`) so the event loop isn't frozen while it runs. For genuinely CPU-heavy work (image processing, ML inference), use a **process pool** instead — Python's Global Interpreter Lock (GIL) means threads don't give true parallelism for CPU-bound work, but separate processes do.

16.6 Context variables and middleware gotchas

Tying back to Chapter 7: request-scoped data set by middleware (like the authenticated user) must be stored using `contextvars` in async code, not plain module-level globals. On a single-threaded event loop juggling many interleaved requests, a naive global variable can leak one request's data into another's response — a subtle, hard-to-reproduce bug class specific to async frameworks.

16.7 Cancellation and timeouts

Always set explicit timeouts on external I/O (database queries, HTTP calls, LLM API calls) — without one, a single hung dependency can tie up a task indefinitely, quietly consuming resources. Use `asyncio.wait_for(...)` or the timeout configuration built into your HTTP/DB client.

Chapter 16 Summary

- The event loop achieves high concurrency by never idling on I/O waits, not by running code in parallel
- A single blocking call inside async code stalls every other concurrent request on that server
- Thread pools escape blocking I/O; process pools escape the GIL for CPU-bound work
- Always set explicit timeouts on external calls — a hung dependency otherwise ties up resources indefinitely

Interview cheat sheet: If asked why async is faster for I/O-bound APIs, the precise answer is “it eliminates idle wait time on I/O, not that it parallelizes computation” — this distinction (Concurrency vs Parallelism) is frequently probed directly.

Common mistake: Using a synchronous HTTP/DB client inside an `async def` function — one of the most common real bugs in FastAPI codebases, and worth mentioning explicitly if you're describing a FastAPI-based design.

Chapter 17: Observability

17.1 The three pillars

Pillar	What it captures	Question it answers
Logging	Discrete, timestamped events	“What exactly happened?”
Metrics	Aggregated numeric measurements over time	“How is the system behaving overall?”
Tracing	The path of a single request across multiple services	“Where did <i>this specific</i> request spend its time?”

17.2 Structured logging and correlation IDs

Log as machine-parseable JSON rather than free-text — this makes logs queryable at scale instead of requiring regex archaeology. Critically, thread a **correlation ID / request ID** through every service a request touches, so all related log lines across an entire distributed call chain can be joined together for one request.

17.3 Metrics and why percentiles beat averages

Prometheus scrapes and stores time-series metrics; **Grafana** visualizes them as dashboards. The metrics that matter most: RPS/QPS, latency percentiles, error rate, and resource saturation.

Warning: An average latency actively hides problems. If p50 is 50ms but p99 is 3000ms, that average might report a healthy-looking ~80ms — while 1% of your users are having a genuinely broken experience the average completely conceals. Production teams watch p95/p99/p999, not the mean.

17.4 Distributed tracing

Jaeger and **OpenTelemetry** give each request a trace ID, and each service hop creates a timed “span” within that trace. The resulting waterfall view shows exactly where time was spent across a multi-service call chain — this becomes essential (not optional) the moment you’re in microservices territory (Chapter 13), since a single user request may touch a dozen services and there’s no other way to know which one is actually slow.

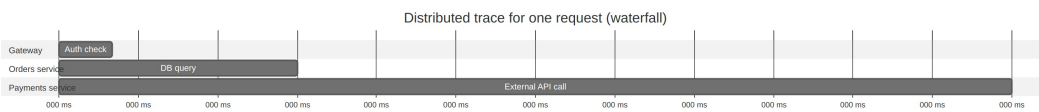


Diagram 15

17.5 Health checks: liveness vs readiness

- **Liveness probe** — “is this process alive at all?” A failure triggers a restart.
- **Readiness probe** — “is this instance ready to receive traffic *right now*?” A failure pulls it from the load balancer’s pool temporarily (e.g. still warming a local cache), without restarting it.

Kubernetes distinguishes these deliberately: conflating them means a temporarily-busy-but-healthy instance gets needlessly killed instead of just being paused from traffic.

Chapter 17 Summary

- Logging, metrics, and tracing answer three different questions and are not substitutes for each other
- Correlation IDs are what make logs useful across a distributed system — without them, joining related events is nearly impossible

- p99 (not average) is the number that reflects real user pain in a system with any meaningful traffic volume

Interview cheat sheet: Mentioning observability unprompted, specifically for any microservices design, is a strong signal — many candidates only think of it when explicitly asked.

Common mistake: Reporting only average latency in a design discussion — always default to mentioning p95/p99 instead, or in addition.

Chapter 18: System Design Patterns

18.1 Circuit Breaker

Prevents cascading failure by “tripping open” after N consecutive failures calling a downstream dependency — subsequent calls fail immediately (fast) for a cooldown period, instead of continuing to hammer a struggling service and making its recovery harder.

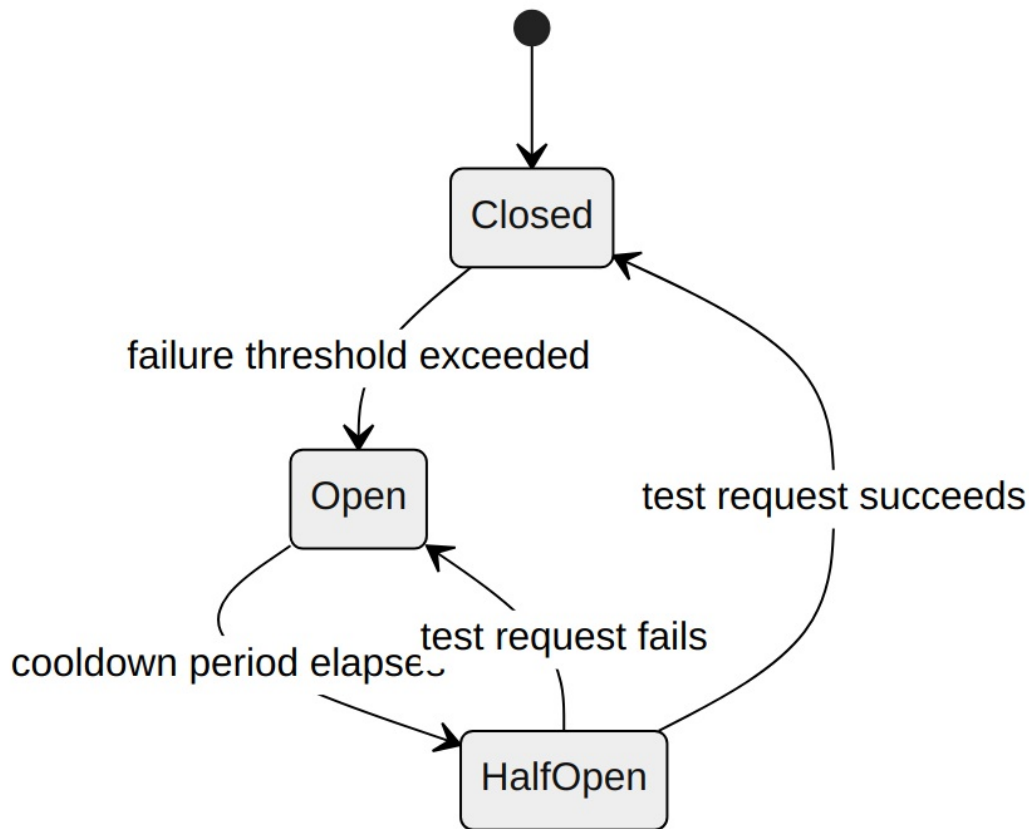


Diagram 16

18.2 Bulkhead

Isolates resources (thread pools, connection pools) per downstream dependency, so one slow or failing dependency can't exhaust resources needed to serve requests to unrelated, healthy dependencies. Named after ship compartments designed to contain flooding to one section.

18.3 Retry with Exponential Backoff (and jitter)

Retry a failed call with increasing delay between attempts, plus randomized jitter, rather than retrying immediately — this avoids many clients retrying in lockstep and overwhelming a service that's just starting to recover.

18.4 Idempotency

Designing an operation so performing it multiple times has the exact same effect as performing it once — typically via a client-generated idempotency key the server tracks. This is what makes “retry on failure” safe rather than dangerous (e.g. safely retrying a payment charge without risking a double charge).

18.5 CQRS (Command Query Responsibility Segregation)

Separates the write model (commands) from the read model (queries), often backed by different, independently optimized data stores — e.g. a normalized relational write store plus a denormalized, cache-friendly read store kept in sync asynchronously.

18.6 Event Sourcing

Stores the full sequence of state-changing events as the source of truth, rather than just the current state — current state is derived by replaying events from the beginning (or from a snapshot). Pairs naturally with CQRS, and gives you a complete audit trail for free.

18.7 Outbox Pattern

Solves the “dual write” problem: writing to a database *and* publishing an event to a queue must be atomic, but they’re two separate systems. The fix: write the event to an “outbox” table within the *same database transaction* as the business data, then a separate relay process reliably publishes from that table to the actual message queue.

18.8 Leader Election and Distributed Locking

Leader election (via Raft, Paxos, or ZooKeeper) lets a set of distributed nodes agree on one coordinator, avoiding conflicting concurrent actions. **Distributed locking** (Redis Redlock, ZooKeeper locks) coordinates exclusive access to a shared resource across multiple processes — genuinely difficult to get fully correct under all failure modes, so use a battle-tested library rather than hand-rolling one.

18.9 Bloom Filter

A space-efficient probabilistic data structure that answers “definitely not present” or “possibly present” for a set membership check — never a false negative, occasionally a false positive. Commonly used to cheaply avoid unnecessary database lookups for keys that don’t exist (directly fixing the cache penetration problem from Chapter 9).

Chapter 18 Summary

- Circuit breaker and bulkhead both exist to contain failure — one by stopping calls to a struggling dependency, the other by isolating resource pools per dependency
- Idempotency is the practical foundation that makes retries (and at-least-once message delivery) safe
- The Outbox pattern is the standard, correct fix for the dual-write problem — worth knowing by name

Interview cheat sheet: If your design includes any retry logic, proactively mention idempotency in the same breath — interviewers listen for whether you connect the two.

Common mistake: Hand-rolling a distributed lock instead of using an established library or service — this is one of the easiest-looking, hardest-to-get-fully-correct problems in distributed systems.

Chapter 19: Design a Large-Scale System

19.1 A repeatable framework

Apply this same sequence to any system design prompt:

1. **Clarify requirements** (Chapter 2) — functional, then non-functional, then explicitly rank the NFRs
2. **Capacity estimation** — rough back-of-envelope math: users, requests/sec, storage growth per day
3. **High-level API design** — the handful of endpoints that define the contract
4. **Data model and database choice** (Chapters 10–11)
5. **Caching layer** (Chapter 9) — what's hot, what's cacheable, what isn't
6. **Async/queue needs** (Chapter 12) — what can happen off the critical path
7. **Scaling strategy** (Chapters 3, 14) — where the bottleneck will appear first, and how you'd address it
8. **Failure scenarios** — what breaks, and what happens to the user when it does
9. **Diagram** — the architecture, tying every piece above together

19.2 URL Shortener (condensed)

Requirements and the core request flow were covered in Chapter 2's worked example. To complete it: use a **key-value store** (Redis or DynamoDB) for the short-code → URL mapping, since lookups are pure O(1) point reads with a heavy read:write skew. Short codes are generated either via a base62-encoded auto-incrementing counter (simple, globally unique, but reveals creation order) or a hash of the URL with collision handling (no shared counter to coordinate, but needs a collision-check step). Given the ~100:1 read:write ratio established in Chapter 2, the caching layer from Chapter 9 (cache-aside, long TTL) absorbs nearly all read traffic.

19.3 Chat Application (WhatsApp-style)

Functional: 1:1 and group messaging, delivery/read receipts, media attachments, online presence. **Non-functional:** very low delivery latency (sub-200ms), message ordering *within* a conversation, at-least-once delivery with client-side deduplication (idempotency, Chapter 18).

The core architectural challenge, unlike most CRUD systems: the server must **push** messages to recipients, so a persistent connection (WebSocket) replaces simple request/response HTTP. And a recipient could be connected to *any* of your N gateway server instances — so when a message arrives, the sending server needs to know which specific gateway instance is holding that recipient's live connection.

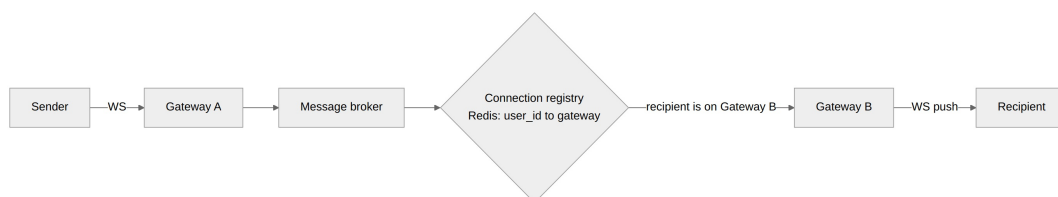


Diagram 17

- **Connection registry** (Redis): maps `user_id → gateway_instance_id`, updated on connect/disconnect
- **Message broker** (Kafka): fans messages out reliably between gateway instances
- **Message store**: a write-heavy, append-mostly workload — Cassandra (Chapter 11) is a common real-world choice
- **Presence**: short-TTL heartbeat keys in Redis; absence of a recent heartbeat = offline
- **Failure scenario**: recipient's gateway crashes mid-message — the broker retains the message (Chapter 12), and it's redelivered once the client reconnects to a new gateway and re-registers

19.4 Video Streaming Platform (Netflix/YouTube-style)

Functional: upload, process into streamable formats, play back at adaptive quality, track watch history. **Non-functional:** massive read/bandwidth scale (video delivery dwarfs almost every other kind of traffic), eventual

consistency is entirely fine for view counts and recommendations.

The single most important architectural decision: application servers must **never** serve raw video bytes. Object storage plus a CDN (Chapter 15) handles that path entirely; app servers only ever serve metadata and API calls.

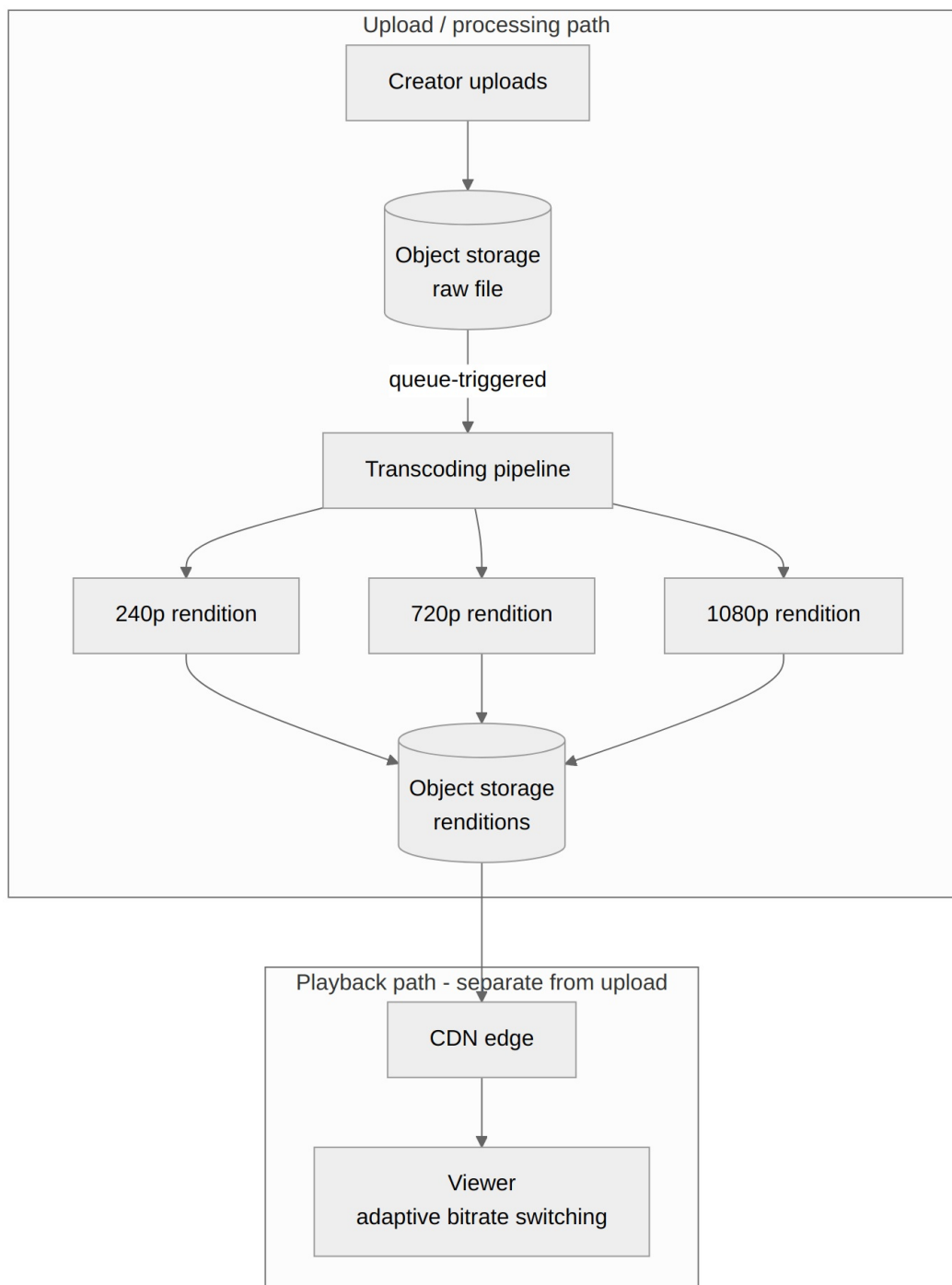


Diagram 18

- **Transcoding** happens asynchronously, triggered by a queue event on upload, producing multiple bitrate renditions (adaptive bitrate streaming via HLS/DASH) so the player can switch quality based on the viewer's real-time bandwidth
- **Metadata** (title, description, view count) lives in a separate database from the video bytes entirely — decoupling a tiny, frequently-updated record from a massive, immutable file
- **Failure scenario:** a transcoding job fails — it's retried via the queue's retry mechanism (Chapter 12); the video simply isn't available yet rather than corrupting anything already published

19.5 Notification System

Functional: send push/email/SMS notifications, respect user channel preferences, support batching/digests. **Non-functional:** eventual consistency is fine, but delivery must be reliable (at-least-once with idempotency) and must never overwhelm a single user (per-user rate limiting, Chapter 6).

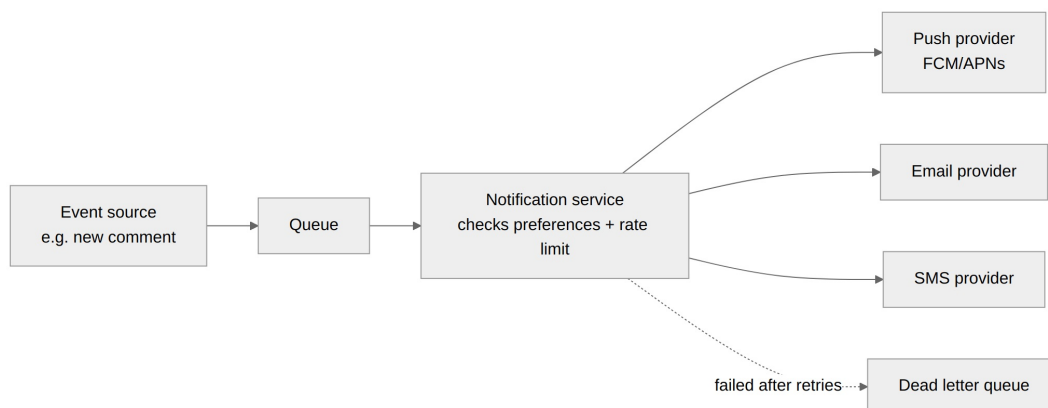


Diagram 19

- Any part of the system publishes a “notification requested” event to a queue — decoupling notification logic from the many features that trigger it
- The notification service consumes, checks the user’s channel preferences and per-user rate limit, then dispatches to the appropriate channel-specific provider
- Failed sends retry with backoff (Chapter 18); after exhausting retries, they land in a DLQ for inspection rather than being silently dropped

19.6 Framework applied to the remaining systems

Rather than full walkthroughs, here’s the core architectural insight for each — enough to reason about them yourself, or ask me to expand any one to full depth:

- **Uber** — fundamentally a real-time geospatial matching problem. Drivers stream location updates over a persistent connection; nearby-driver lookup uses geospatial indexing (geohashing or quad-trees) rather than scanning all drivers; dispatch is a continuous matching algorithm, not a simple database query.
- **Twitter/X feed** — the defining trade-off is **fan-out-on-write** (precompute every follower’s timeline when a tweet is posted — fast reads, expensive for high-follower accounts) vs **fan-out-on-read** (assemble the timeline at read time — cheap writes, expensive reads). Production systems use a hybrid: fan-out-on-write for typical accounts, fan-out-on-read for celebrity accounts with millions of followers (the “celebrity problem”).
- **Instagram Feed** — same fan-out trade-off as Twitter, plus it’s CDN-heavy for media delivery (Chapter 15) in a way text-only feeds aren’t.
- **File Storage (Dropbox/Drive)** — large files are split into chunks; identical chunks across different files/users are deduplicated via content hashing (massive storage savings); sync introduces conflict resolution when the same file changes on two devices before either syncs.
- **AI Chat Application** — directly relevant to your own RAG chatbot work: responses stream token-by-token over Server-Sent Events or WebSocket rather than waiting for the full completion; conversation history needs a defined context-window management strategy (truncate, summarize, or retrieve-relevant-only); calls to the underlying LLM provider need the queueing and rate-limiting patterns from Chapter 6; RAG specifically adds a vector database in the read path for retrieval before generation.
- **Voice AI Platform** — the system your VoiceOS architecture work maps onto most directly: real-time audio needs a low-latency transport (WebRTC, not plain HTTP), the STT → LLM → TTS pipeline needs each stage streaming into the next rather than waiting for full completion at each step to keep perceived latency low, and multi-turn voice sessions need explicit session state management (Chapter 1’s stateful-service problem, here unavoidable) since the conversation context must persist across an ongoing audio stream, plus orchestration across whatever agent pipeline sits behind it.

Chapter 19 Summary

- Every system design prompt reduces to the same nine-step framework — the specific architecture changes, the process of getting there doesn't
- The recurring meta-pattern across nearly every "famous" system: separate the hot, high-volume path (messages, video bytes, audio) from the metadata/control path, and let each scale independently
- Fan-out-on-write vs fan-out-on-read is one of the most frequently tested trade-offs across feed-shaped systems specifically

Interview cheat sheet: When given an unfamiliar system to design, explicitly narrate the framework as you go ("first let me nail down requirements, then estimate scale...") — interviewers are evaluating your process at least as much as your final diagram.
